

How to Build a 24/7 AI Trading Agent with Claude Code and Alpaca

Use Claude Code Routines and the Alpaca API to build an autonomous trading agent that researches markets, places trades, and journals its decisions without manual intervention.

What This Agent Actually Does

Most AI trading agents end up in one of two places: a rigid rule-based bot that cannot adapt, or a one-shot Claude session that makes a few trades and then stops. Neither is enough.

What you want is a Claude Code agent that runs on a schedule, scans the market, reasons about what to do, places trades through the Alpaca API, and logs every decision in a structured journal.

The agent has three core jobs:

Research — Pull market data, news, and technical signals for a defined watchlist.

Trade — Use that research to place buy, sell, or hold orders through Alpaca.

Journal — Record each decision, including when the agent chooses to do nothing.

This setup uses paper trading by default. You can switch to live trading later, once you trust its behavior.

Prerequisites

Before writing any code, you need three things.

1. An Alpaca Account

Alpaca offers commission-free stock and crypto trading through a REST API. Create an account and generate both paper and live API keys. Keep them separate, and use paper keys during development.

Your `.env` file should look like this:

```
APCA_API_KEY_ID=your_paper_key_id
APCA_API_SECRET_KEY=your_paper_secret_key
APCA_BASE_URL=https://paper-api.alpaca.markets
```

2. Claude Code Installed and Configured

Claude Code needs to be running locally, with scheduled routines enabled. That scheduling layer is what makes the agent autonomous instead of manually triggered.

3. A Basic File Structure

```
/trading-agent
  CLAUDE.md          # Agent instructions and persona
  watchlist.json     # Stocks/ETFs to monitor
  journal/           # Trade logs
  scripts/
    research.py      # Data fetching helpers
    trade.py         # Order placement helpers
  .env               # API credentials (never commit this)
```

You do not need to build everything at once. Start with the structure, then add research, execution, and journaling step by step.

Step 1: Define the Agent's Persona in `CLAUDE.md`

`CLAUDE.md` is the agent's operating manual. Claude Code reads it at the start of every session, so this is where you define its behavior and constraints.

A strong starting template looks like this:

```
# Trading Agent Instructions

You are an autonomous trading agent managing a paper portfolio.

## Core Responsibilities
- Every market day at 9:45 AM ET: Run the research routine
- Every market day at 10:00 AM ET: Evaluate research and place trades
- Every market day at 4:15 PM ET: Write a journal entry covering the day

## Rules You Must Always Follow
- Never invest more than 5% of total portfolio value in a single position
- Never place market orders; always use limit orders within 0.2% of ask
- If a position drops 8% from entry, close it immediately
- Always write a journal entry, even on days with no trades
- Never place trades when the market is closed

## Decision Framework
Before placing any trade, answer these questions:
1. What is the current portfolio cash balance?
2. What positions are already open?
3. What does recent news say about this ticker?
4. What do the 20-day and 50-day moving averages show?
5. What is the downside if this trade fails?

## Output Format
Every action must be logged to `journal/YYYY-MM-DD.md` in a structured format.
```

The more specific this file is, the more predictable the agent becomes. Vague instructions produce vague behavior.

Step 2: Build the Research Skill

The agent needs real market data before it can make decisions. This helper script fetches prices, account data, positions, and news.

```
# scripts/research.py

import os
import requests
import json

ALPACA_KEY = os.getenv("APCA_API_KEY_ID")
ALPACA_SECRET = os.getenv("APCA_API_SECRET_KEY")
BASE_URL = os.getenv("APCA_BASE_URL")

def get_bars(symbol, timeframe="1Day", limit=60):
    """Fetch historical price bars for a symbol."""
    headers = {
        "APCA-API-KEY-ID": ALPACA_KEY,
        "APCA-API-SECRET-KEY": ALPACA_SECRET,
    }
    url = f"https://data.alpaca.markets/v2/stocks/{symbol}/bars"
    params = {
        "timeframe": timeframe,
        "limit": limit,
        "adjustment": "raw"
    }
    response = requests.get(url, headers=headers, params=params)
    return response.json()

def get_account():
    """Get current portfolio status."""
    headers = {
        "APCA-API-KEY-ID": ALPACA_KEY,
        "APCA-API-SECRET-KEY": ALPACA_SECRET,
    }
    url = f"{BASE_URL}/v2/account"
    response = requests.get(url, headers=headers)
    return response.json()

def get_positions():
    """Get all open positions."""
    headers = {
        "APCA-API-KEY-ID": ALPACA_KEY,
        "APCA-API-SECRET-KEY": ALPACA_SECRET,
    }
    url = f"{BASE_URL}/v2/positions"
    response = requests.get(url, headers=headers)
    return response.json()
```

```

def get_news(symbol):
    """Get recent news for a symbol."""
    headers = {
        "APCA-API-KEY-ID": ALPACA_KEY,
        "APCA-API-SECRET-KEY": ALPACA_SECRET,
    }
    url = "https://data.alpaca.markets/v1beta1/news"
    params = {
        "symbols": symbol,
        "limit": 5,
        "sort": "desc"
    }
    response = requests.get(url, headers=headers, params=params)
    return response.json()

if __name__ == "__main__":
    import sys
    action = sys.argv[1] if len(sys.argv) > 1 else "account"
    symbol = sys.argv[2] if len(sys.argv) > 2 else None

    if action == "bars" and symbol:
        print(json.dumps(get_bars(symbol)))
    elif action == "news" and symbol:
        print(json.dumps(get_news(symbol)))
    elif action == "positions":
        print(json.dumps(get_positions()))
    else:
        print(json.dumps(get_account()))

```

The agent can call this script from the command line. For example, `python scripts/research.py bars AAPL` returns 60 days of price data as JSON. Claude can then compute moving averages, detect trends, and incorporate news into its decision-making.

Step 3: Build the Trade Execution Skill

Placing trades through Alpaca is straightforward. The real challenge is adding guardrails before the agent touches capital.

```

# scripts/trade.py

import os
import requests
import json
import sys

ALPACA_KEY = os.getenv("APCA_API_KEY_ID")
ALPACA_SECRET = os.getenv("APCA_API_SECRET_KEY")
BASE_URL = os.getenv("APCA_BASE_URL")

def place_order(symbol, qty, side, limit_price=None):

```

```

"""Place a buy or sell order."""
headers = {
    "APCA-API-KEY-ID": ALPACA_KEY,
    "APCA-API-SECRET-KEY": ALPACA_SECRET,
    "Content-Type": "application/json"
}

order_data = {
    "symbol": symbol,
    "qty": qty,
    "side": side, # "buy" or "sell"
    "type": "limit" if limit_price else "market",
    "time_in_force": "day",
}

if limit_price:
    order_data["limit_price"] = str(limit_price)

url = f"{BASE_URL}/v2/orders"
response = requests.post(url, headers=headers, json=order_data)
return response.json()

def cancel_all_orders():
    """Cancel all open orders."""
    headers = {
        "APCA-API-KEY-ID": ALPACA_KEY,
        "APCA-API-SECRET-KEY": ALPACA_SECRET,
    }
    url = f"{BASE_URL}/v2/orders"
    response = requests.delete(url, headers=headers)
    return response.status_code

def get_market_status():
    """Check if the market is open."""
    headers = {
        "APCA-API-KEY-ID": ALPACA_KEY,
        "APCA-API-SECRET-KEY": ALPACA_SECRET,
    }
    url = f"{BASE_URL}/v2/clock"
    response = requests.get(url, headers=headers)
    return response.json()

if __name__ == "__main__":
    action = sys.argv[1]

    if action == "status":
        print(json.dumps(get_market_status()))
    elif action == "order":
        symbol = sys.argv[2]
        qty = sys.argv[3]
        side = sys.argv[4]
        limit_price = sys.argv[5] if len(sys.argv) > 5 else None
        print(json.dumps(place_order(symbol, qty, side, limit_price)))
    elif action == "cancel":
        print(cancel_all_orders())

```

Step 4: Set Up the Trade Journal

Every decision the agent makes should be logged. This is non-negotiable. Without a journal, you cannot audit behavior or improve the system.

Use a simple markdown format:

```
# Trade Journal - 2026-04-18

## Portfolio Status
- Cash: $12,450.00
- Positions: NVDA (42 shares @ $845.20), SPY (15 shares @ $521.00)
- Total Value: $23,891.80

## Market Research

### NVDA
- 20-day MA: $838.50 | 50-day MA: $812.00 - bullish trend intact
- News: Positive analyst upgrade from Morgan Stanley, +8% PT increase
- Earnings: 3 weeks out - potential catalyst

### AAPL
- 20-day MA: $195.20 | 50-day MA: $198.80 - short-term weakness
- News: Supply chain concerns in Taiwan Strait reporting
- Decision: No action, watch for stabilization

## Trades Executed
| Time | Symbol | Action | Qty | Price | Reasoning |
|-----|-----|-----|-----|-----|-----|
| 10:03 | NVDA | BUY | 5 | $847.50 | MA trend + analyst upgrade = entry |

## Positions Closed
None today.

## End-of-Day Reflection
NVDA trade aligned with thesis. Held off on AAPL given macro uncertainty in news.
Tomorrow: Watch AAPL for reversal signal, check MSFT earnings preview.
```

The agent should write this file automatically at the end of each trading day. Over time, the journal becomes a valuable feedback loop you can use to refine future decisions.

Step 5: Configure the Claude Code Routine

This is the step that makes the agent truly autonomous. Claude Code Routines let you schedule tasks in headless mode, so the agent runs without manual intervention.

Create `.claude/routines.json` in your project directory:

```
{
  "routines": [
    {
      "name": "Morning Research",
      "schedule": "45 9 * * 1-5",
      "timezone": "America/New_York",
      "prompt": "Run the morning research routine. Check market status first. If the market is open, pull bars and news for every symbol in watchlist.json. Summarize findings in a structured format and save to journal/YYYY-MM-DD.md under the Research section.",
      "allowed_tools": ["bash", "read", "write"]
    },
    {
      "name": "Trading Session",
      "schedule": "0 10 * * 1-5",
      "timezone": "America/New_York",
      "prompt": "Run the trading session. Read today's research from the journal file. Check current positions and cash balance. For each symbol in the watchlist, decide whether to buy, sell, or hold based on the research and the trading rules in CLAUDE.md. Place limit orders for any decisions. Log all actions and reasoning to the journal.",
      "allowed_tools": ["bash", "read", "write"]
    },
    {
      "name": "End of Day Journal",
      "schedule": "15 16 * * 1-5",
      "timezone": "America/New_York",
      "prompt": "Run the end-of-day routine. Pull final positions and account value. Read all orders placed today. Complete the journal entry for today with a reflection section. Note what worked, what didn't, and what to watch tomorrow.",
      "allowed_tools": ["bash", "read", "write"]
    }
  ]
}
```

The 1-5 in the cron schedule means Monday through Friday only, so the agent will not run on weekends.

Even on weekdays, each session begins by calling `python scripts/trade.py status` to verify that the market is open. If it is a holiday or there is a delayed open, the agent logs the condition and exits cleanly.

Step 6: The Watchlist

Keep the watchlist small at first. The agent needs enough symbols to make good decisions, but not so many that it becomes noisy or overextended.

```
{
  "watchlist": [
    {
      "symbol": "SPY",
      "description": "S&P 500 ETF – baseline market exposure",
      "max_allocation_pct": 15
    },
    {
      "symbol": "QQQ",
      "description": "Nasdaq ETF – tech sector exposure",
      "max_allocation_pct": 10
    },
    {
      "symbol": "NVDA",
      "description": "GPU/AI infrastructure – high conviction holding",
      "max_allocation_pct": 8
    },
    {
      "symbol": "AAPL",
      "description": "Large cap tech – stability anchor",
      "max_allocation_pct": 8
    },
    {
      "symbol": "MSFT",
      "description": "Cloud/enterprise – AI infrastructure play",
      "max_allocation_pct": 8
    }
  ],
  "cash_reserve_pct": 20
}
```

The `max_allocation_pct` field sets a hard cap for each position. The `cash_reserve_pct` field forces the agent to keep 20% in cash, even when it is bullish. That prevents it from becoming overallocated across the portfolio.

Step 7: Risk Controls That Actually Work

Risk management fails in two common ways: the rules are too vague, or they exist only in the prompt and get ignored under pressure.

Use three layers of protection:

1. CLAUDE.md Instructions

Write clear, numeric rules. Avoid vague language like “be careful” or “manage risk.”

2. Script-Level Validation

Validate every order before it reaches Alpaca:

```
def validate_order(symbol, qty, side, current_price, account_value,
current_positions):
    """Pre-flight checks before placing any order."""
    order_value = qty * current_price
    allocation_pct = (order_value / account_value) * 100

    if allocation_pct > 10:
        return False, f"Order exceeds 10% allocation limit:
{allocation_pct:.1f}%"

    total_invested = sum(p['market_value'] for p in current_positions)
    if (total_invested + order_value) / account_value > 0.80:
        return False, "Order would violate 20% cash reserve requirement"

    return True, "Order validated"
```

3. Alpaca's Built-In Safeguards

Use paper trading first, then rely on Alpaca's buying power checks and other platform protections as a final layer of defense.

Step 8: Running Multiple Agents in Parallel

Once the single-agent setup is stable, you can add a second agent to review decisions. This creates a simple multi-agent debate pattern.

The structure is straightforward:

- **Agent A (Trader)** — Generates trade ideas and proposes orders
- **Agent B (Risk Reviewer)** — Checks those proposals and approves or rejects them

This adds a second layer of judgment before anything is executed.

Agent B runs 15 minutes after Agent A and reads the journal file. If it disagrees with a proposed trade, it flags the entry and sets `review_required` to true. The execution routine will not place flagged orders until a human clears them.

This adds latency, but it catches cases where Agent A sounded convincing and was still wrong. It is the same basic idea as parallel agent teams sharing a task list: independent review produces better decisions than a single agent acting alone.

For most people, the single-agent setup is the right place to start. Learn how it behaves on paper trading before adding more complexity.

Step 9: Managing Token Costs

A trading agent that runs three times a day, five days a week, across five symbols can burn through tokens quickly. Control the budget from the start.

A few practical rules:

- Pull only the data you need. Sixty bars is usually enough for daily decisions.
- Summarize prior context instead of passing full journals. Use a summary file, not 30 full entries.
- Set explicit tool budgets. Cap tool calls per session so the agent does not spiral into endless research.

The key is to define what “done” looks like for each routine. Once the agent has enough information to decide, it should stop.

Step 10: Monitoring Without Babysitting

The point of this system is that it runs on its own, but you still need visibility.

Two simple checks help a lot:

Daily Email Digest

Add a final step to the end-of-day routine that sends the day’s journal by email. A simple SendGrid or Mailgun script is enough.

```
# scripts/notify.py – sends today's journal as an email
import os, sys
import sendgrid
from sendgrid.helpers.mail import Mail

def send_digest(journal_path):
    with open(journal_path, 'r') as f:
        content = f.read()

    sg = sendgrid.SendGridAPIClient(os.getenv("SENDGRID_API_KEY"))
    message = Mail(
        from_email="agent@yourdomain.com",
        to_emails=os.getenv("NOTIFY_EMAIL"),
        subject=f"Trading Agent Report – {journal_path.split('/')[-1]}",
        plain_text_content=content
    )
    sg.send(message)
```

```
if __name__ == "__main__":  
    send_digest(sys.argv[1])
```

Heartbeat Check

At the end of each run, the agent writes a `heartbeat.json` file with a timestamp and status. If there is no heartbeat by 5 PM on a trading day, something likely went wrong.

This is the simplest version of the heartbeat pattern: small, reliable, and easy to monitor.

What Realistic Results Look Like

Before setting expectations, look at how similar systems perform in practice. In the OpenClaw AI trading bot challenge, an agent traded a \$10,000 account over 30 days with fully transparent results — including losses.

The takeaways are straightforward:

- AI agents excel at **consistent execution**, not market prediction.
- The **journal is the most valuable asset** — it's where you uncover why the agent underperforms.
- **Paper trading overstates performance.** Live trading introduces slippage, partial fills, and execution friction.

Treat this system as a process you refine over time, not a plug-and-play profit machine.